

Community System

Overview

Communities are Yenkasa's primary geographic and interest-based segmentation layer. They influence registration, feed scope, messaging relevance, and notification targeting.

Current implementation

Community creation and membership

- users select up to two communities at registration
- communities are country-scoped and must be approved before use
- community creation is gated by role and verification rules
- join and leave behavior updates both the user and the community record

Community visibility

- public route: `GET /api/communities/public`
- authenticated route: `GET /api/communities`
- pending review route: `GET /api/communities/pending`

Community notifications

- approved community posts trigger batched notifications to eligible members
- privacy rules can exclude blocked or muted recipients

Important modules

- `routes/community.routes.js`
- `models/community.model.js`
- `services/communityPostNotification.service.js`
- `helpers/community.helper`

Known issues

- `memberCount` can drift and is recomputed in some read flows
- membership state exists both as community-owned `members` and user-owned `joinedCommunities`
- some per-user fields in the community model (`isJoined`, `canPost`, `canModerate`) are view concerns stored at schema level

Scaling concerns

- notification fan-out for community posts is membership-size dependent
- joins and leaves can require follow-up count repairs

Recommended improvements

1. treat membership as one canonical relation and derive counters from it
2. move view-only user-specific flags out of the persisted community schema
3. shift large community notification fan-out to a queue-backed worker